Plane Detection Algorithm

Currently being developed by Tori E. Field

Purpose

The plane detection algorithm is a step in the process of automating calibration of Zephyr meteor radar network. The plane detection algorithm is a Python function which automatically identifies airplanes in RTI data. Eventually, this algorithm will be used in tandem with other tools developed to reduce human involvement in the calibration of Zephyr, with the goal of more quickly and accurately calibrating Zephyr.

The plane detection algorithm is able to determine when a plane is visible to Zephyr, and provide preliminary information about what times that plane is visible, and what gates to look for it in.

Breakdown

The plane detection algorithm can be thought of as the combination of two main functions. The CFAR algorithm, and the DBSCAN algorithm. The CFAR algorithm is used to find individual detection events, and the DBSCAN algorithm clusters these detection events into detected objects. Due to the persistence of planes, each plane detection corresponds to numerous individual detection events.

Preprocessing

The quality of data passed into the plane detection algorithm can greatly effect it's success. The function detect_plane requires an xarray input, which can be generated in many different ways. In general the function reduce_xarray which is part of the Zephyr repository will be used, as it is very efficient at highlighting useful data, and suppressing noise. Other methods of prep can be used such as PCA, but ultimately, the better input to the function, the more accurate it will output results.

CFAR

The CFAR algorithm uses individual timestamps, and evaluates for detection events across all gates at each individual timestamp. Each detection event is a tuple containing the gate and time of the detection, as well as some additional information. The collection of these detection events are then passed into DBSCAN.

DBSCAN

DBSCAN is a clustering algorithm based primarily on density of points. The DBSCAN algorithm clusters individual detection events into detected objects based on the density of points, and their proximity to each other. It is also effective at eliminating outliers, or detection events which are not correlated to any object.

TODO: add links

Data Preparation

Before RTI data can be processed in any form it must be loaded into the workspace from a file. In this example glob and pickle are used. These steps are based on code written by James Monaco.

```
In [64]: import glob # A common system tool for finding and parsing files
         data_dir = './rti_examples' # The name of the folder with the files
```

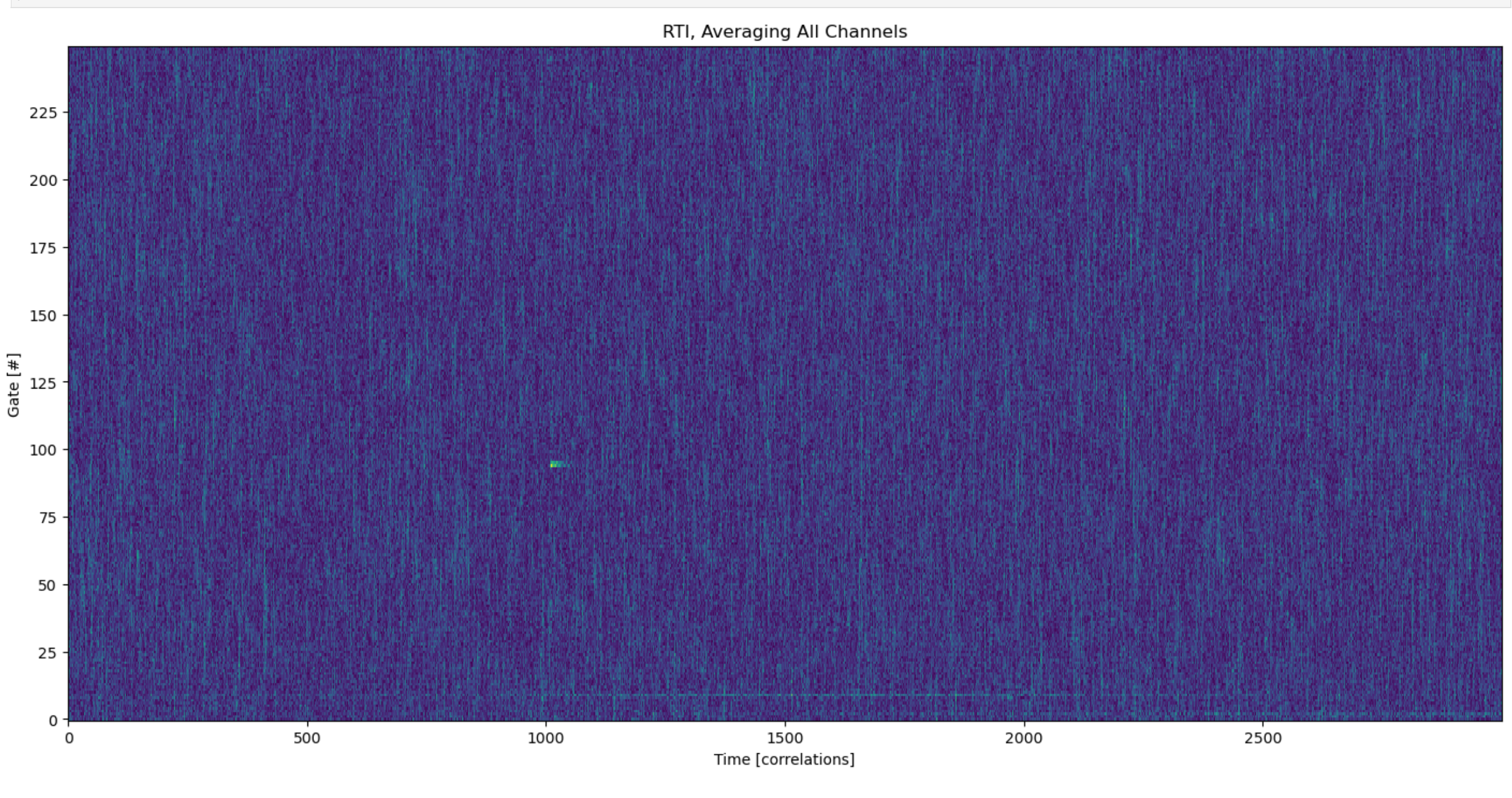
```
In [65]: import pickle # For loading files

         file_name = 'RTI_Plane_2meteors.rti'
         with open(f'{data_dir}/{file_name}', "rb") as f:
                 rti_xarray = pickle.load(f)
```

Using this method, RTI data is loaded into the workspace as rti_xarray, with data type xarray.dataarray (https://docs.xarray.dev/en/latest/generated/xarray.DataArray.pad.html). The xarray type is comparable to a numpy.array (https://numpy.org/doc/2.1/reference/generated/numpy.array.html) but with some changed and additional functionality. In this case rti_xarray is a four dimensional array (gate, time, channel, doppler shift) with complex values. Despite doppler shift being it's own dimension, it only takes on one value in this example.

Once loaded, different techniques can be used both to prepare rti_xarray for use in the plane detection function, and to visualize it's contents. In almost all cases, the complex values of the array must be converted to real values. This is done using the numpy.abs (https://numpy.org/doc/2.1/reference/generated/numpy.absolute.html) but other techniques can be used with varying results. Complex values may not cause immediate issues, as many libraries automatically preform this conversion, but will cause errors in the function detect_planes, the detection algorithm.

One way to visualize and prep this 4 dimensional data is to average the doppler shift, and channel dimensions, reducing it to a 2D array:

```
In [66]: import numpy as np
         import matplotlib.pyplot as plt # Plotting
         from matplotlib.ticker import MaxNLocator

         # RTI diagram
         rti_array = rti_xarray.values[:, 0, :250, :]
         fig, ax = plt.subplots(1, 1, figsize=(14, 7), layout='constrained')
         rti_for_plot = np.abs(rti_array.mean(0))  # Collapse Doppler Dimension, mean across channels

         ax.imshow(rti_for_plot, origin='lower', aspect='auto', interpolation='none', extent=None)
         ax.set_xlabel('Time [correlations]')
         ax.set_ylabel('Gate [#]')
         ax.yaxis.set_major_locator(MaxNLocator(integer=True))
         ax.set_title('RTI, Averaging All Channels')
         plt.show()
```



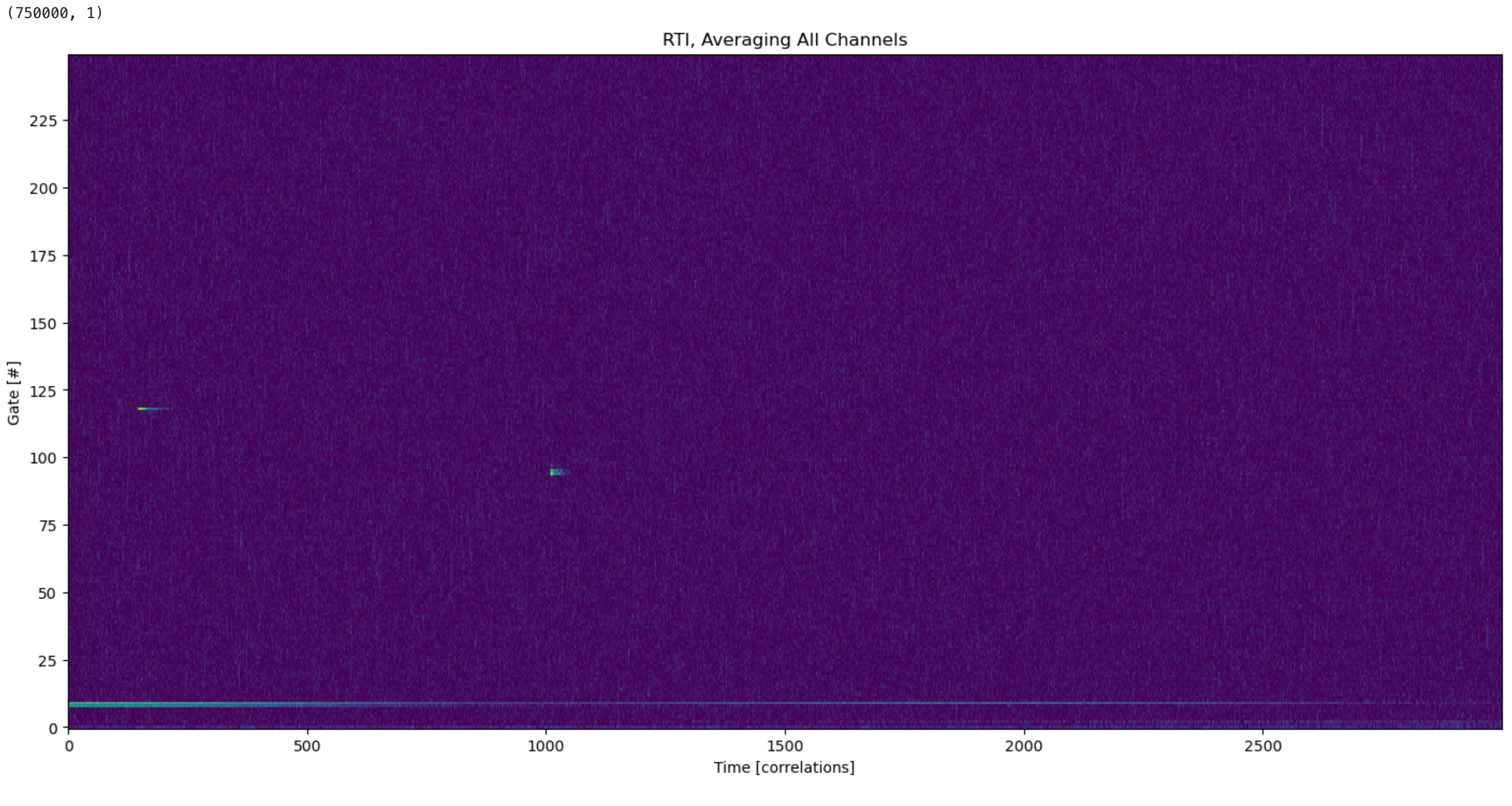
This RTI data contains a plane and two meteors, but only one meteor is visible, and the plane is only barely visible. This RTI data spans approximately 15s across 1000 gates (indexed at 0) but for plotting purposes only the first 250 gates are shown. Virtually all meteors will be within the first 250 gates with planes only in the first 20-25, many still below 10. Despite this, the full RTI data should be fed into detect_plane

This RTI data which spans approximately 15s across 1000 gates (indexed at 0) has been reduced to only the first 250 gates, which is more than necesary to view meteors. Planes, are much lower maybe reaching gates 20 or 25, many still below gate 10. Note here that data should be fed into the plane detection whole, or indexing values will be wrong.

For a clearer view, PCA can be used which essentially highlights data peaks, such as those caused by planes and meteors, while reducing the effects of background noise.

```
In [67]: from sklearn.decomposition import PCA

         # Performing PCA
         n_ch, n_gates, n_times = rti_array.shape
         X = np.moveaxis(rti_array, 0, -1).reshape(-1, n_ch)
         X = abs(X)
         pca = PCA(n_components=1)
         pca.fit(X)
         rti_PCA = pca.fit_transform(X)
         print(rti_PCA.shape)
         rti_PCA = np.reshape(rti_PCA, (n_gates, n_times, 1)).squeeze()

         # Plotting
         fig, ax = plt.subplots(1, 1, figsize=(14, 7), layout='constrained')
         pca_for_plot = np.abs(rti_PCA[:, :])
         ax.imshow(pca_for_plot, origin='lower', aspect='auto', interpolation='none', extent=None)
         ax.set_xlabel('Time [correlations]')
         ax.set_ylabel('Gate [#]')
         ax.yaxis.set_major_locator(MaxNLocator(integer=True))
         ax.set_title('RTI, Averaging All Channels')
         plt.show()
```

```
(750000, 1)
```



In this case, both meteors are visible, and the plane is very prominent.

Unfortunately, PCA is slow, and while great for visualization, its speed makes it difficult to use prepping large quantities of data. The zephyr repository includes functions which are used to prepare RTI data for use in the meteor detection algorithm. The plane detection algorithm has been designed to work with input from these functions specifically. Primarily the outputs reduce_xarray can be directly input into detect_plane. For ease, both reduce_xarray and parse_selection are included here.

```
In [68]: def parse_selection(selection): #Function from Zephyr
             """
             Parse selection for slices or ranges, with support for 'all' and specific indices or ranges.

             Parameters:
             - selection: Can be 'all', a string for channel names, or a list of indices.
             """
             # If selection is "all", return a slice that selects everything
             if isinstance(selection, str) and selection == "all":
                 return slice(None)  # Select all elements

             # If selection is a string like "120:-1", handle as a slice
             elif isinstance(selection, str) and ":" in selection:
                 try:
                     return slice(*map(lambda x: int(x) if x.strip() else None, selection.split(":")))
                 except ValueError as e:
                     raise ValueError(f"Invalid slice string: '{selection}'. Error: {e}")

             # If selection is a list or array of values, return it as-is
             elif isinstance(selection, (list, np.ndarray)):
                 return selection

             else:
                 raise ValueError(f"Invalid selection format: '{selection}'.")
```

```
In [69]: def reduce_xarray(xarray_in, dim_operations, value_agg_func=np.abs): #Function from zephyr
             """
             Reduces the RTI data based on the provided dimension operations (aggregation, selection).

             Parameters:
             - dim_operations (dict): Specifies "agg_func" and "selection" for each dimension.
                 dim_operations = {
                     "tx_rx_channel_pair": {"agg_func": None, "selection": "all"},  # Select polarimetric channels
                     "doppler_hz": {"agg_func": None, "selection": [0]},  # Slice specific range
                     "gate": {"agg_func": None, "selection": "all"},  # No aggregation
                     "time": {"agg_func": None, "selection": "all"}  # Median over specific time slice
                     }
             - value_agg_func (callable): Function to transform the data (e.g., np.abs, np.angle).

             Returns:
             - reduced_data (numpy.ndarray): Reduced data (2D or 3D) ready for plotting.
             """
             xarray_out = xarray_in.copy()
             xarray_out.attrs =  xarray_in.attrs.copy()

             # Apply operations for each dimension
             for dim, operations in dim_operations.items():
                 if dim not in xarray_out.dims:
                     raise ValueError(f"Dimension '{dim}' not found in data.")

                 # Apply selection
                 selection = operations.get("selection", "all")
                 if dim == "tx_rx_channel_pair" and selection == "polarimetric":
                     polarimetric_values = xarray_out.coords['polarimetric_channel'].values
                     desired_channels = polarimetric_values[polarimetric_values != None]
                     mask = np.isin(polarimetric_values, desired_channels)
                     xarray_out = xarray_out.sel(tx_rx_channel_pair=mask)
                 else:
                     parsed_selection = parse_selection(selection)
                     xarray_out = xarray_out.sel({dim: parsed_selection})

                 # Apply aggregation if specified
                 agg_func = operations.get("agg_func", None)
                 if agg_func is not None:
                     if not callable(agg_func):
                         raise ValueError(f"Aggregation function for '{dim}' must be callable or None.")
                     xarray_out = xarray_out.reduce(agg_func, dim=dim, keep_attrs=True)

             # Apply value aggregation function (e.g., np.abs, np.angle)
             if value_agg_func is not None:
                 if not callable(value_agg_func):
                     raise ValueError("value_agg_func must be a callable function.")
                 xarray_out = value_agg_func(xarray_out)

             # Modify metadata attributes
             if value_agg_func is not None:
                 if 'units' in xarray_out.attrs:
                     xarray_out.attrs['units'] = f'{xarray_out.attrs["units"]} ({value_agg_func.__name__})'
                 else:
                     xarray_out.attrs['units'] = value_agg_func.__name__

             # Return reduced data (converted to numpy array)
             return np.squeeze(xarray_out)
```

Running the reduce_xarray function results in a 2 dimentional xarray containing only gate and time axies, shown below.

```
In [70]: dim_operations = {
                             "tx_rx_channel_pair": {"agg_func": np.std, "selection": "all"},
                             "doppler_hz": {"agg_func": None, "selection": [0]}}
         rti_for_det = reduce_xarray(rti_xarray, dim_operations, value_agg_func=np.abs) #Set up Data in the same format as in repository
```

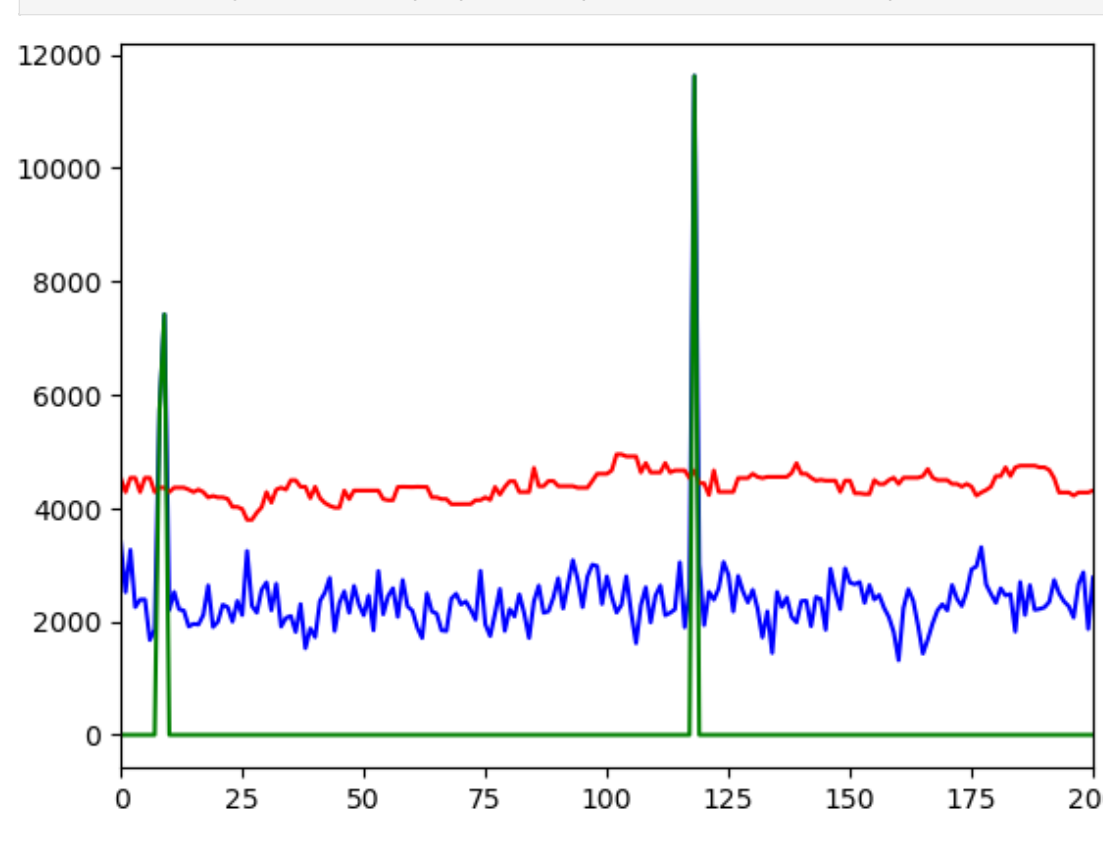
After being properly formated using reduce_xarray data can be fed into the plane detection algorithm. Other methods such as averaging, or PCA shown above are also valid, with their own benefits and drawbacks. No mater the data preperation methods, plane detection function requires a 2D xarray (gate like, time like) and will cause errors if given a numpy.array as input.

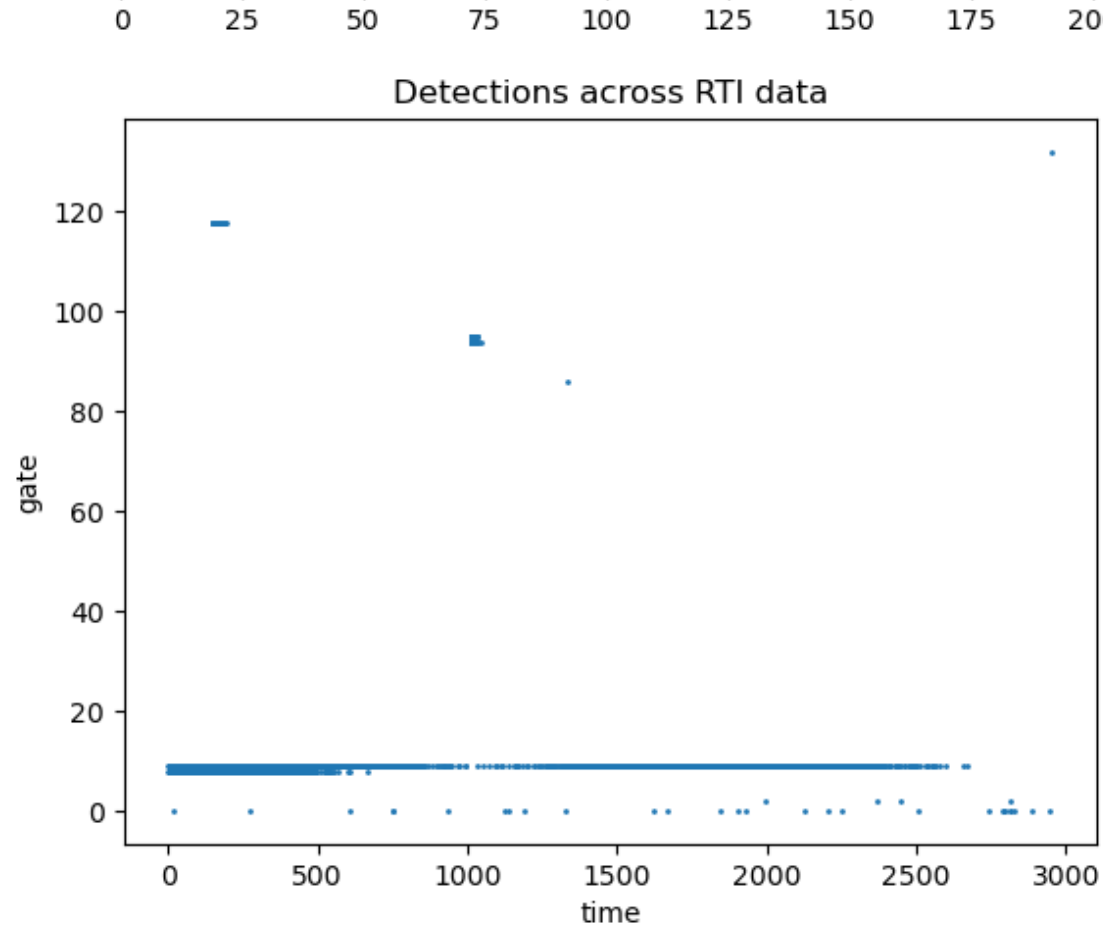
Examples of Algorithm in Use

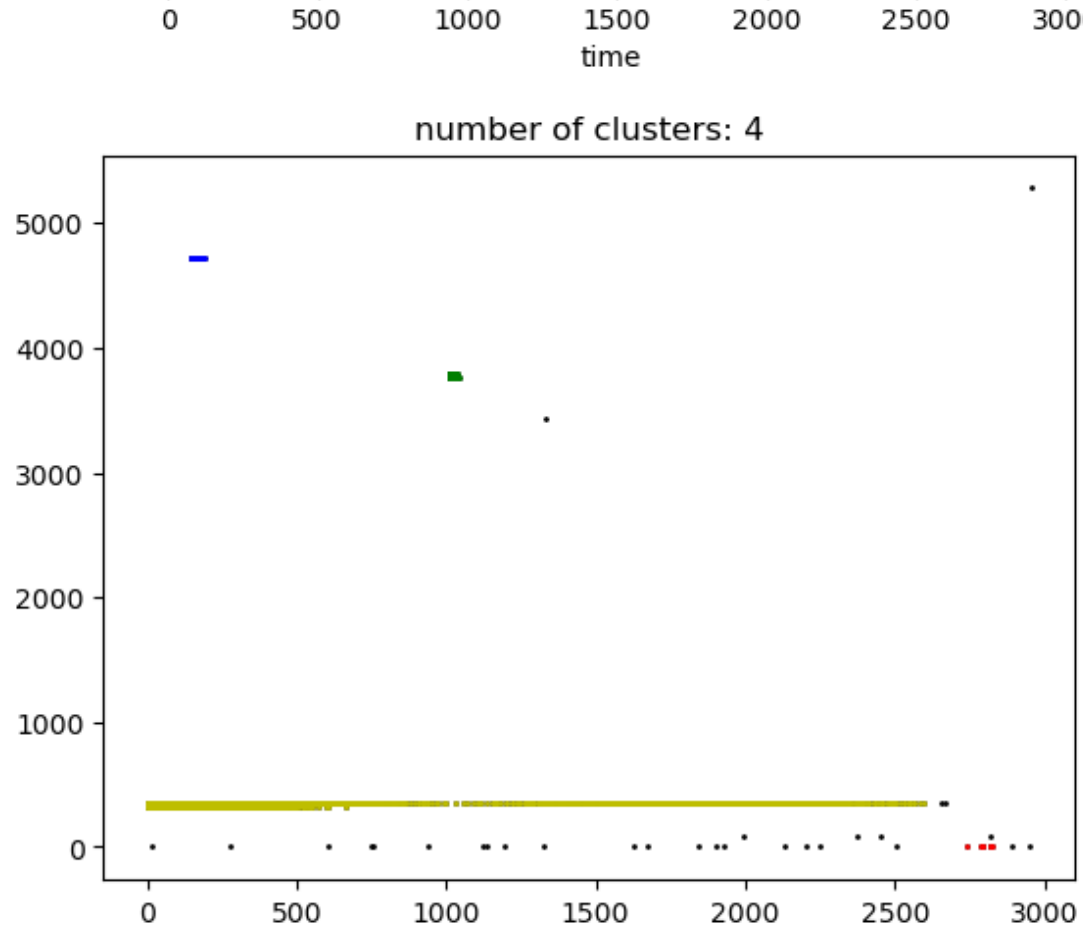
Example with Basic Data and Inputs

Here the plane detection algorithm will be run with default settings on rti_for_det, from Data Preparation. The algorithm will default to settings which have already been tuned, which work on a wide array of RTI data. Default options can be selected by leaving the detect_options dictionary empty. Additionally, there are three optional plots, which show steps in the detection process. For this example, all three are shown and discussed.

```
In [71]: import Plane_Detection as PDA #Import plane detection algorithm as PDA
         detections,img,list_dict_properties,peak_idx=PDA.detect_plane(rti=rti_for_det,start_samples=2000,detect_options={},optional_plots=[150,'CFAR_plot','plot_detections',"DE
```







The first plot, CFAR_plot shows the rti data, targets, and calculated threshold for a single timestamp of data. The timestamp should be specified by entering an integer as the first value of optional_plots. It can be used to help tune the main CFAR algorithm which governs which points are are considered targets.

The Second graph,plot_detections, shows detection events across all the RTI data. These are individual points which the CFAR algorithm labeled as targets or individual detection events. These detections may include noise, and must still be grouped into object detection with DBSCAN.

The third graph, DBSCAN, shows the data after being clustered by DBSCAN into detected objects. Each detected object is depicted in a different color, with individual detection events not belonging to a detected object in black. Notice that both the meteors and the plane are considered separate detected objects. Additionally, there is a 4th detected object in the first gate due to ground interference, discussed later.

list_dict_properties is a list of dictionaries containing information on each detected object, such as which gates and times it is visible.

```
In [72]: list_dict_properties
```

```
Out[72]: [{'plane_number': 0,
           'max_time': 2597,
           'min_time': 1,
           'max_gate': 9.0,
           'min_gate': 8.0},
          {'plane_number': 1,
           'max_time': 193,
           'min_time': 147,
           'max_gate': 118.0,
           'min_gate': 118.0},
          {'plane_number': 2,
           'max_time': 1047,
           'min_time': 1010,
           'max_gate': 95.0,
           'min_gate': 94.0},
          {'plane_number': 3,
           'max_time': 2828,
           'min_time': 2741,
           'max_gate': 0.0,
           'min_gate': 0.0}]
```

Function Inputs and Outputs

```
Inputs:
    rti: xarray,
        RTI data of the form [gate-like, time-like]
    start_samples: int,
        Initial sample number, used in finding accurate values of sample_number, and 1d_idx
    detect_options: Dict,
        optional detection parameters, when left blank, function will use default tuning
        num_ref_cells: int,
            number of reference cells
        num_guard_cells: int,
            number of guard cells
        bias: float,
            multiplicitive bias to threshold values
        gate_range: int tuple,
            (min gate, max gate) describing range of rti gates to focus on
        edge_method: string,
            method used to deal with edges in CFAR algorithm. Options: zero_padding, mean_padding, median_padding,
            symetric, wrap, mean, median
        method: numpy function,
            cell averaging method used in CFAR algorithm ex: np.mean, np.min. np.max, np.median
        eps: float,
            Radius used in DBSCAN clustering
        min_samples: int,
            minimum number of points in neighborhood to be considered a cluster
        y_scaling: float,
            Scaling factor along y axis, can be used to preevent planes at close altitudes
            from being identified as a single object
    optional_plots: list,
        [time of interest, 'CFAR_plot','plot_detections','DBSCAN']

Outputs:
    list_detections: list,
        list of tuples: (gate, time, 1D_idx, sample_number)
    img: returns 0
    list_dict_properties: list,
        list containing dictionaries for each plane detection, with basic information on it
    list_peak_idx_1D: list,
        list of idx_1D for each detection event
```

The Default options, are as follows:

detect_options_default = {"num_ref_cells":10,"num_guard_cells":3,"bias":1.9,"gate_range":(0,200),"edge_method":'symetric', "method":np.median,"eps":55,"min_samples":5,"y_scaling":40}

Any of these values can be changed individually, or an entirely new dictionary containing detection algorithms can be used as input.

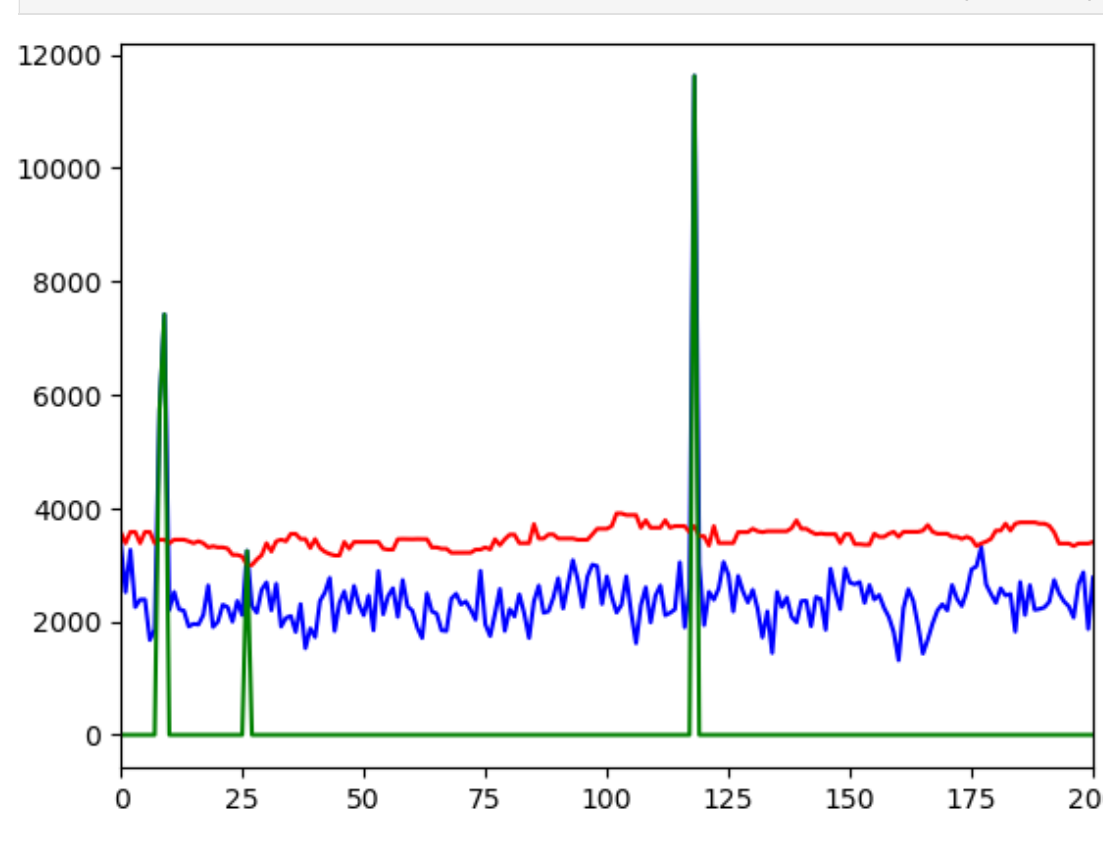
Examples of how changing different values changes detection

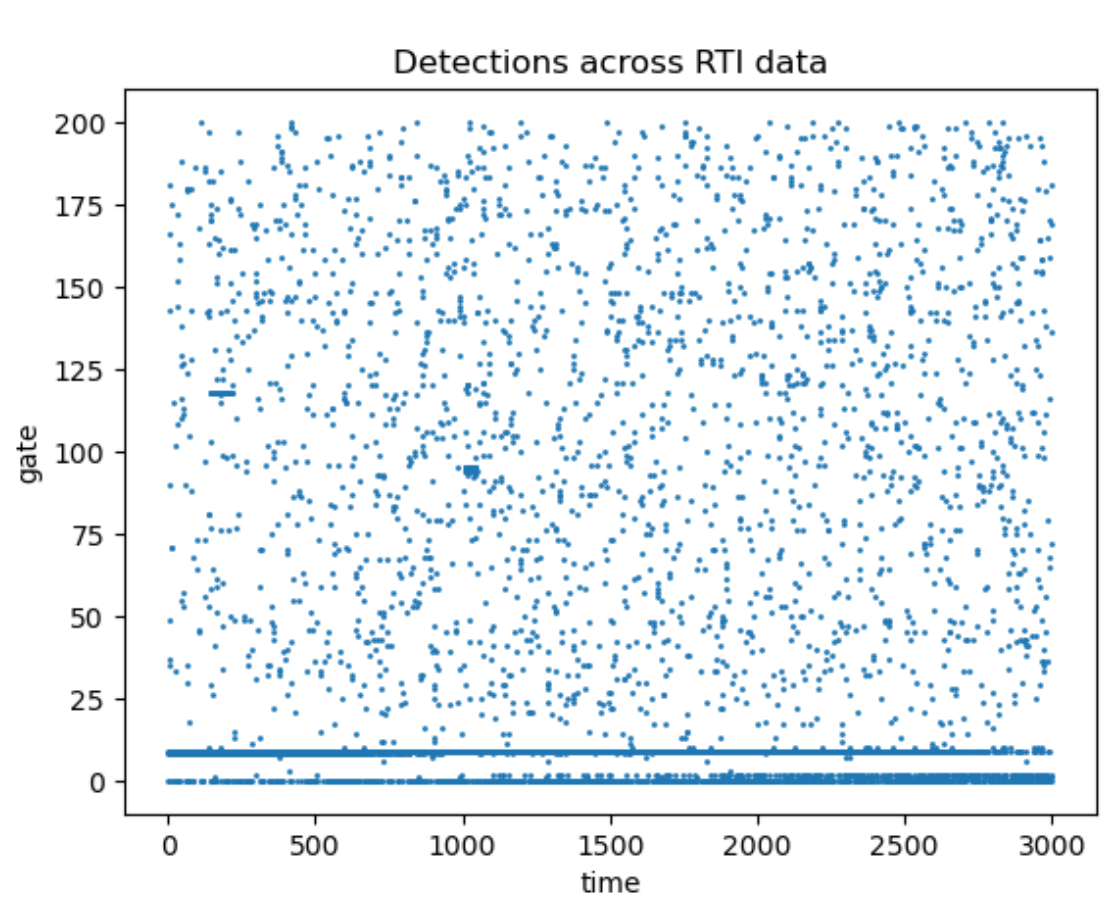
Changing detection options can have drastic effects on the algorithms performance, so it is suggested to use the default options whenever possible. Adjusting only a couple detection options may mandate adjusting the others as well for accurate detection.

Changing Bias

The bias is the multiplicative bias for threshold values, that is when a threshold value is calculated by averaging reference cells, it is multiplied by the bias to create a buffer between the threshold and the data. By default the bias has a value of 1.9 but can be changed by specifying "bias" in detect_options. In this case, the bias is lowered from a default value fo 1.9 to 1.5

```
In [73]: detections,img,list_dict_properties,peak_idx=PDA.detect_plane(rti=rti_for_det,start_samples=2000,
                                                                     detect_options={"bias":1.5},
                                                                     optional_plots=[150,'CFAR_plot','plot_detections'])
```





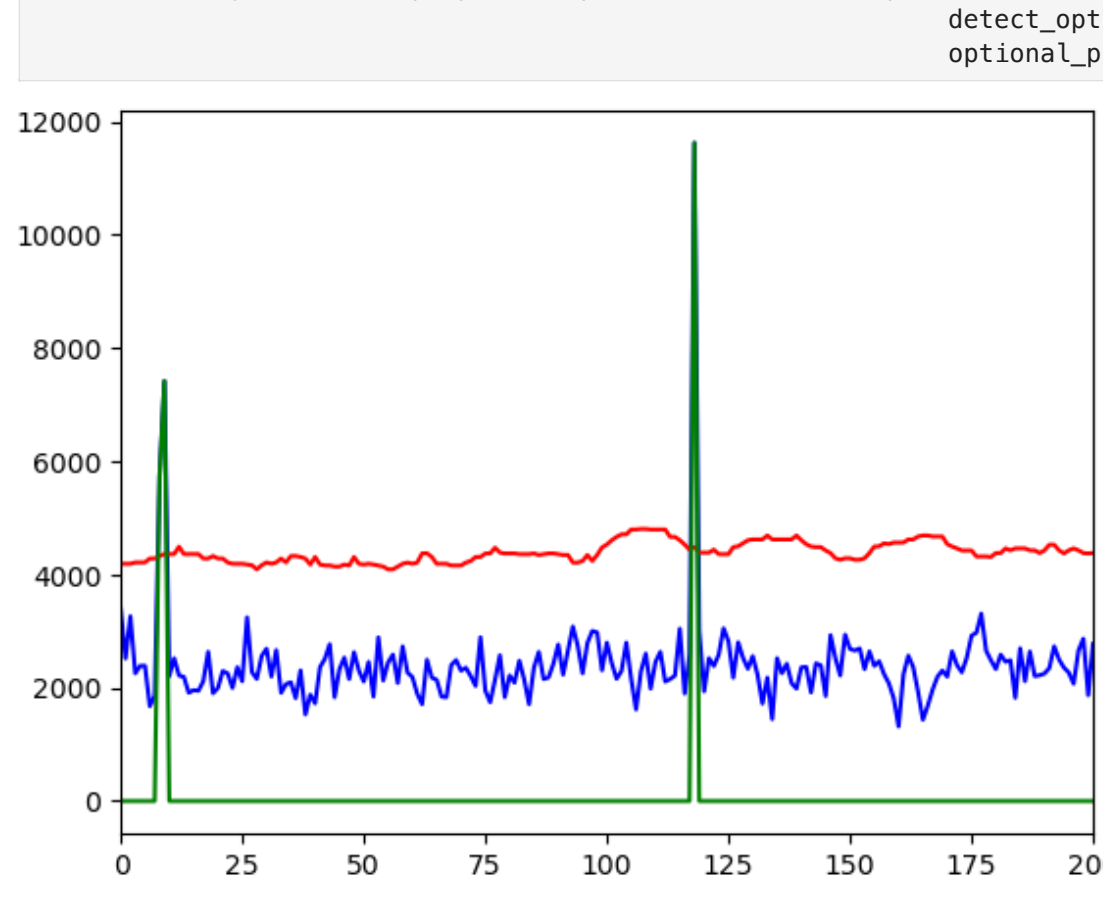
Lowering the bias results in more detections events overall. The location of the plane became much more prominent, but so does the background noise, especially the ground interference. On the contrary, by increasing the bias the number of false detection events are reduced, but legitimate detection events are also more likely to be missed.

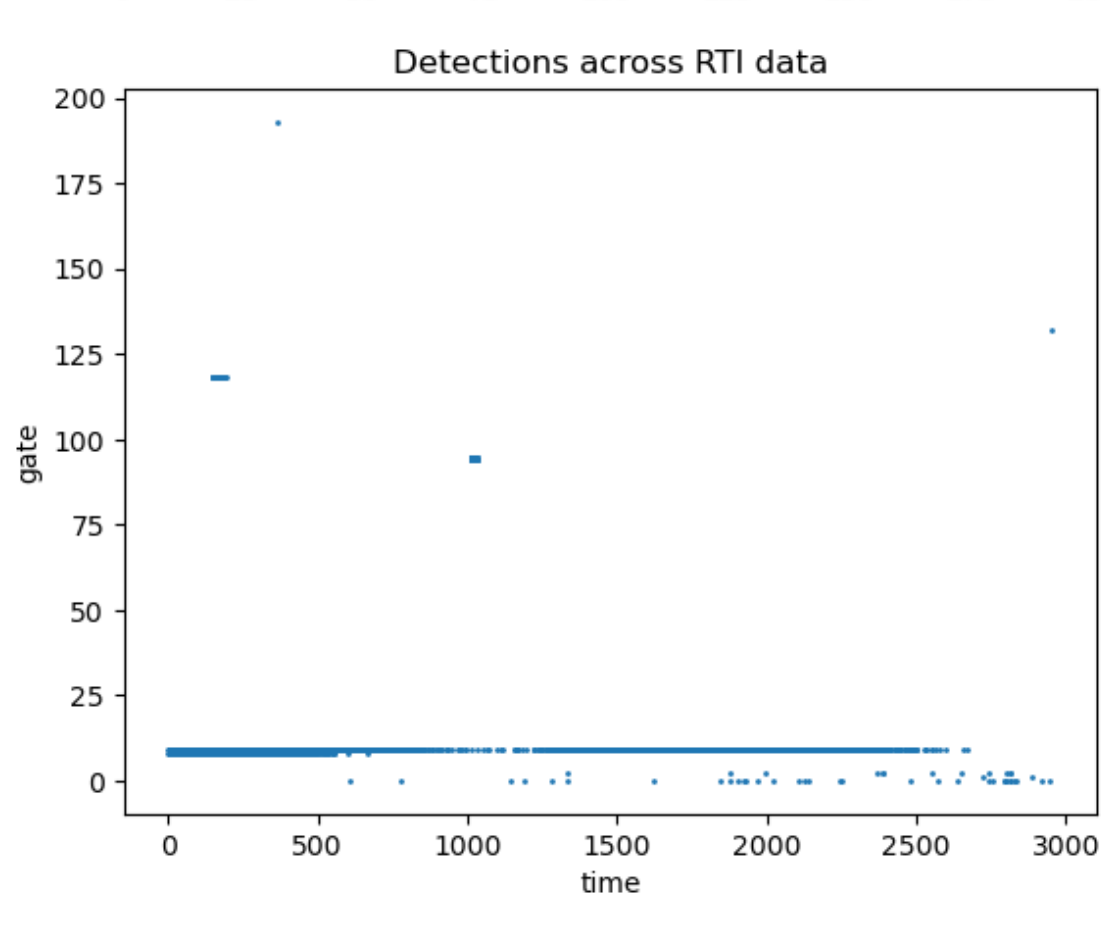
Changing this value can work, but generally requires changing the density requirements for the DBSCAN, such as eps and min_samples.s

Changing the Number of Reference Cells

The number of reference cells, num_ref_cells is the number of cells used in calculating an average at each gate. By default, num_ref_cells=10 but is increased to 20 in this example.

```
In [74]: detections,img,list_dict_properties,peak_idx=PDA.detect_plane(rti=rti_for_det,start_samples=2000,
                                                                     detect_options={"num_ref_cells":20},
                                                                     optional_plots=[150,'CFAR_plot','plot_detections'])
```



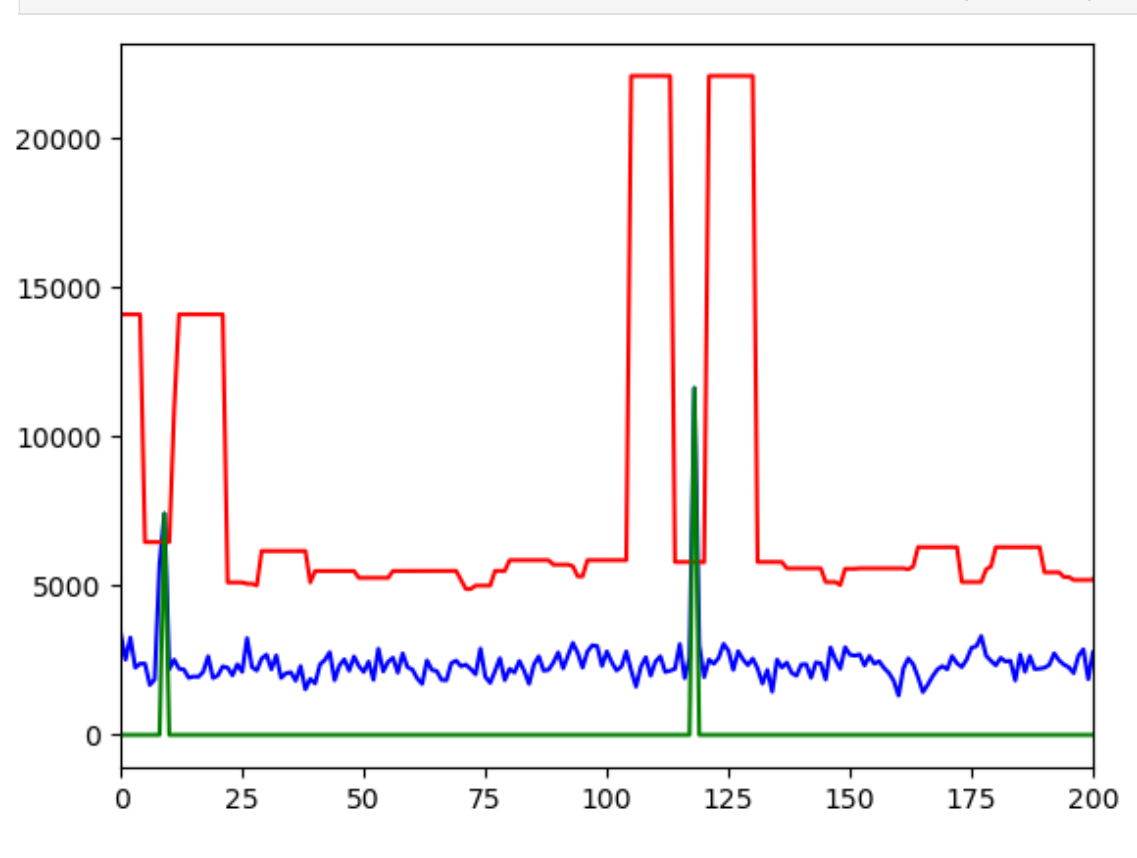


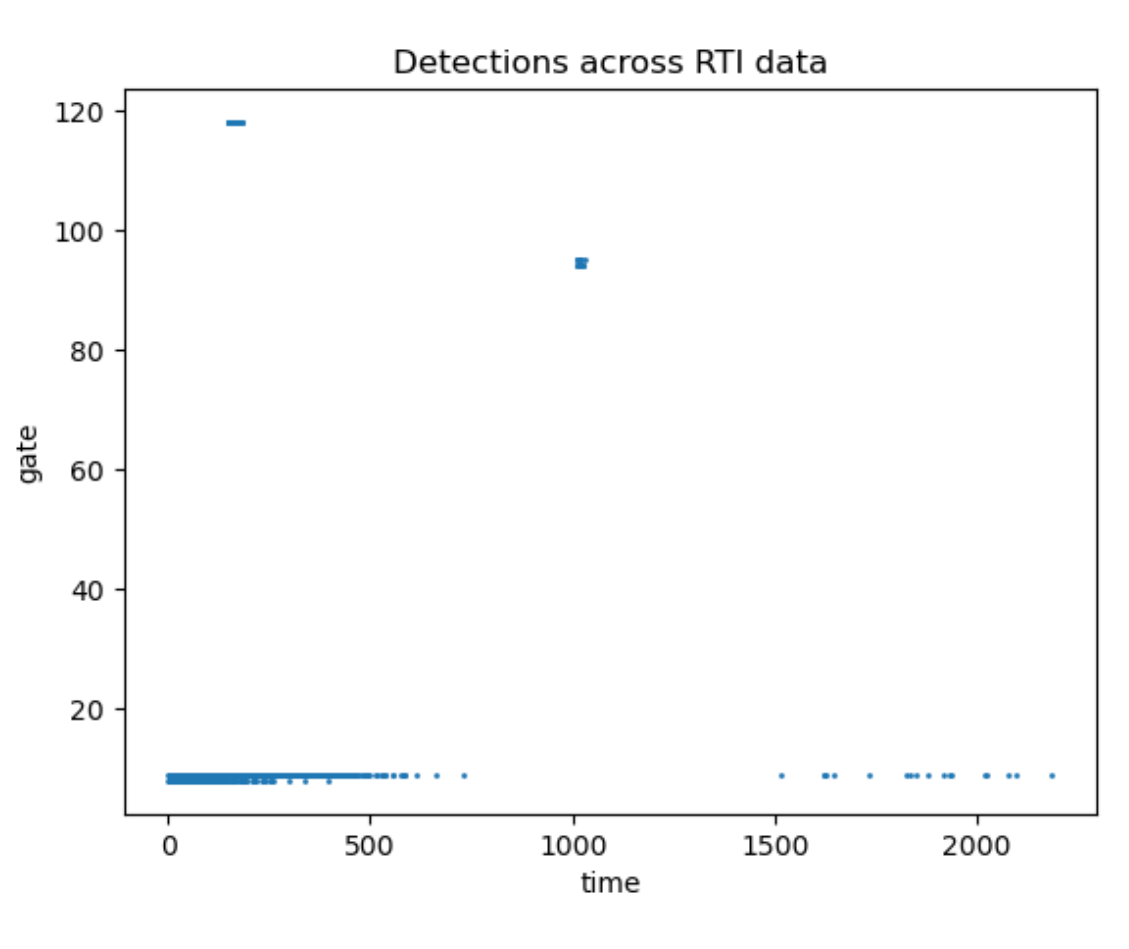
The value of num_ref_cells directly impacts CFAR being one of it's main inputs. It essentially adjusts the smoothness of the threshold value calculated at each timestamp. The more reference cells used, the smoother the threshold value will be and less reference cells will result in a fluctuation threshold value. In this example, increasing the number of reference cells from 10 to 20 results in more false detections across the rti data due to being less able to adapt to the background noise.

Changing Method

The method option determines which method is used in finding a threshold value before applying the bias. By default this option is set to np.median, to calculate the median of the reference cells, but can take values such as np.max, np.min, np.mean. Each of these options will significantly affect how noise spikes effect the value of the threshold. In the example below, the method is set to np.max.

```
In [75]: detections,img,list_dict_properties,peak_idx=PDA.detect_plane(rti=rti_for_det,
                                                                 start_samples=2000,
                                                                 detect_options={"method":np.max},
                                                                 optional_plots=[150,'CFAR_plot','plot_detections'])
```





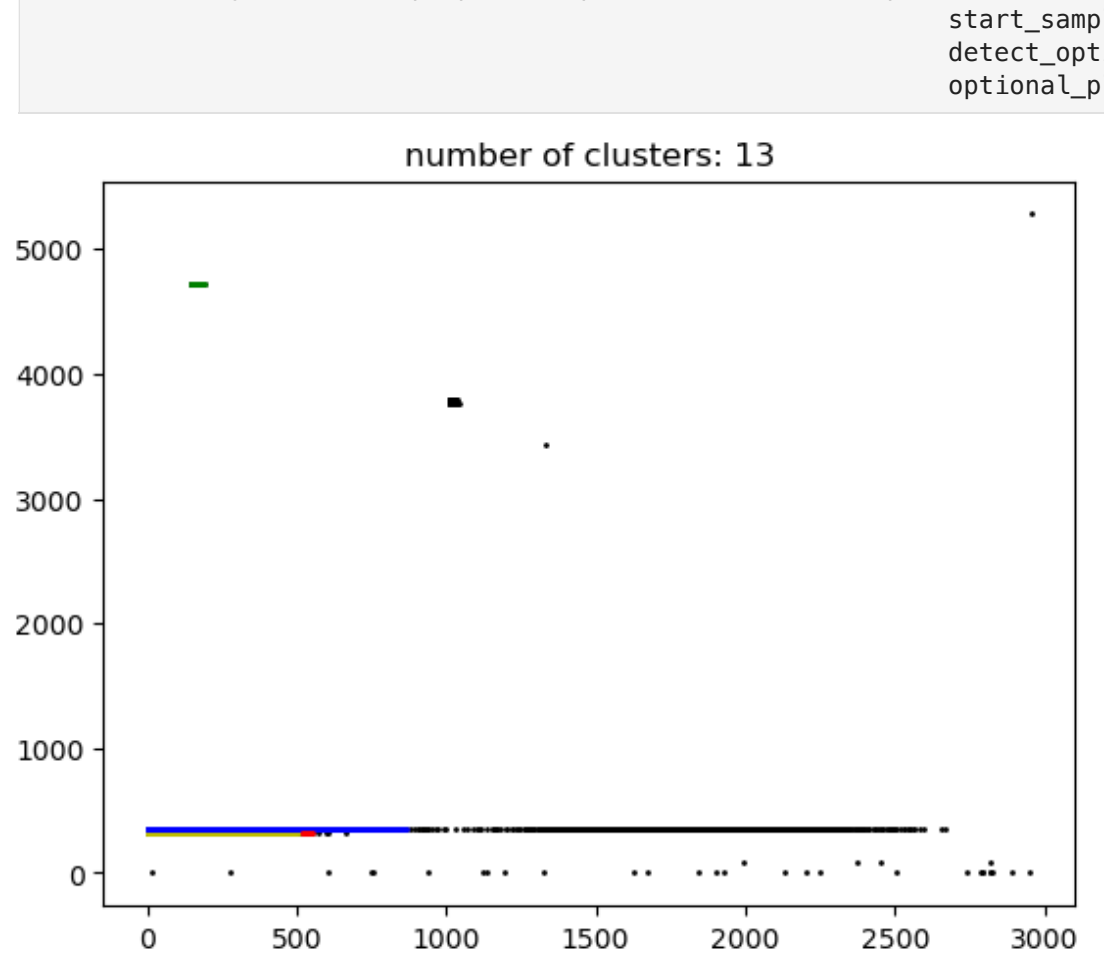
Changing the method can both have impacts on the accuracy of detection events, as well as on the time taken for the algorithm to run. In some cases this could mean trading speed for accuracy.

The CFAR algorithm used is heavily based on the following Youtube video: https://www.youtube.com/watch?v=BEg29UuZk6c More information on the function used, and CFAR generally can be found there.

Changing eps

eps, or epsilon represents the radius of a neighborhood around individual points, if enough points, specified by min_samples fall within that radios, a cluster begins to form, every point within eps of a cluster will be added to that cluster until all points in the dataset are either clusters or noise. By default this value is reduced from 55 to 10.

```
In [76]: detections,img,list_dict_properties,peak_idx=PDA.detect_plane(rti=rti_for_det,
                                                                 start_samples=2000,
                                                                 detect_options={"eps":10},
                                                                 optional_plots=['DBSCAN'])
```



In this case, reducing eps increases the number of detection events classified as noise, as well as drastically increases the number of clusters, or detected objects. Additionally, in this case the value of y_scaling is less than y_scaling (default of 40), the algorithm is completely unable to cluster detection events across different gates into the same object. To prevent this problem, ensure that eps>=y_scaling. For plane detection I have found that the ratio eps/y_scaling should be around 1-2 for best results.

Examples with different datasets

In addition to tuning parameters, types of planes, other objects, and background noise can influence the accuracy of detection. This section discusses several examples, and the points in which they work well and fall short.

"Fading" plane

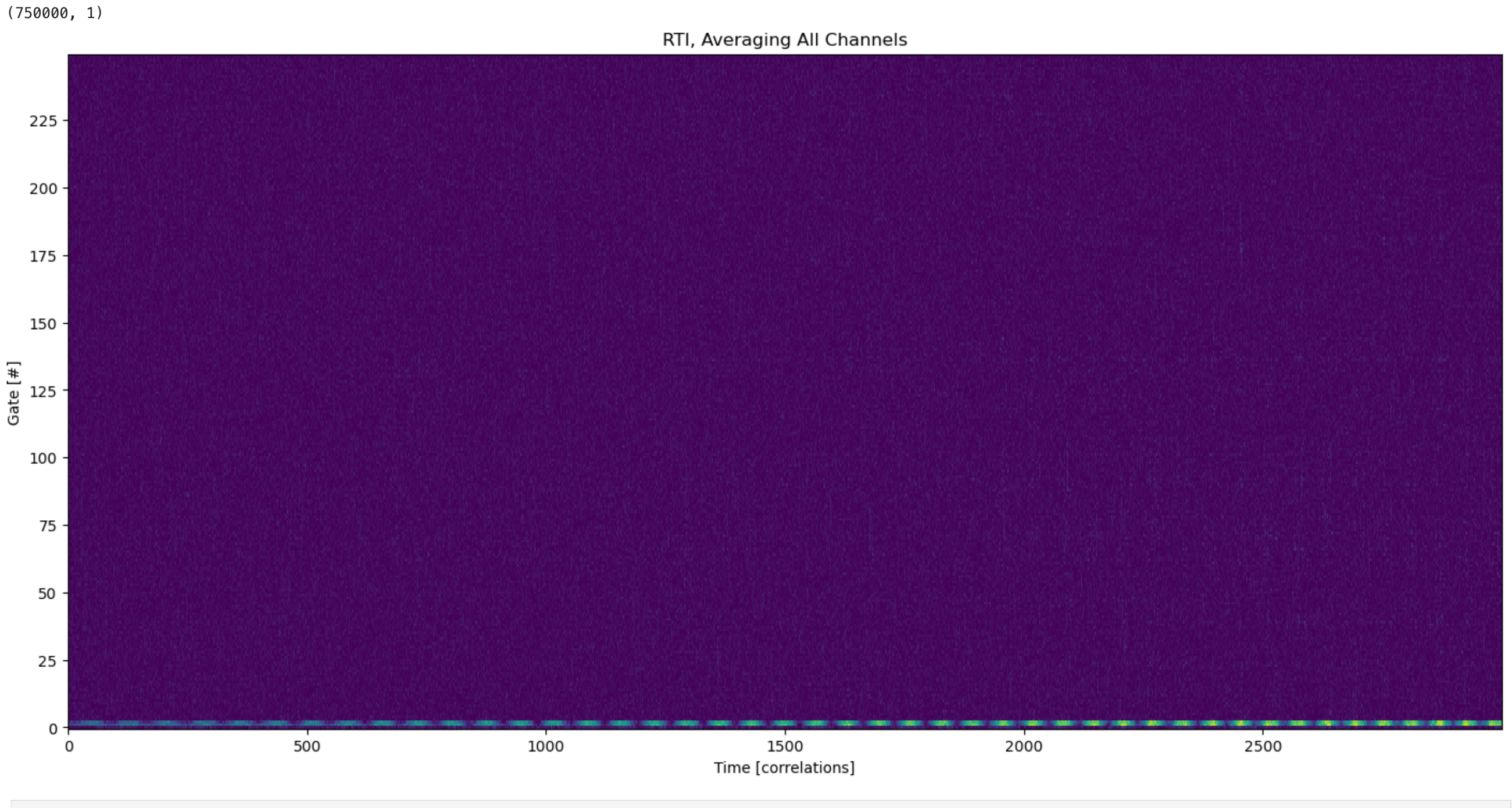
Many planes have exhibited "fading" pattern. They become more or less pronounced over a period of time, called it's phase. Planes with a relatively small period work very well with the plane detection algorithm.

```
In [77]: file_name2 = 'RTI_faiding_plane_15s.rti' #Load in rti with fading plane
         with open(f'{data_dir}/{file_name2}', "rb") as f:
                 rti_xarray_fadingplane = pickle.load(f)
```

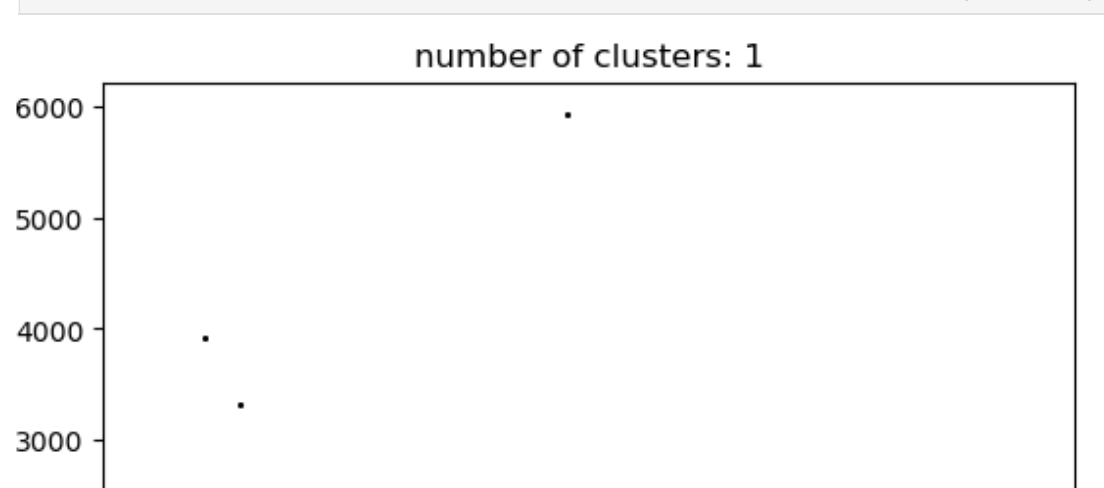
Bellow a PCA plot is shown exhibit the "fading" behavior.

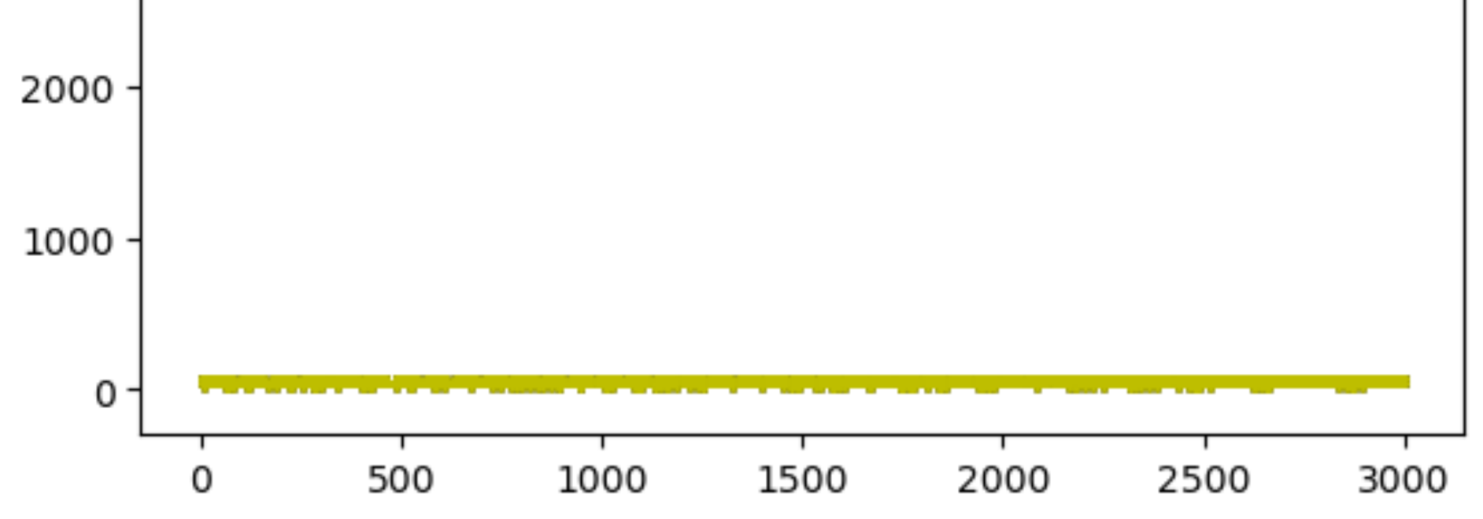
```
In [78]: rti_array_fadingplane = rti_xarray_fadingplane.values[:, 0, :250, :]
         # Performing PCA
         n_ch, n_gates, n_times = rti_array_fadingplane.shape
         X = np.moveaxis(rti_array_fadingplane, 0, -1).reshape(-1, n_ch)
         X = abs(X)
         pca = PCA(n_components=1)
         pca.fit(X)
         rti_PCA_fadingplane = pca.fit_transform(X)
         print(rti_PCA_fadingplane.shape)
         rti_PCA_fadingplane = np.reshape(rti_PCA_fadingplane, (n_gates, n_times, 1)).squeeze()

         # Plotting
         fig, ax = plt.subplots(1, 1, figsize=(14, 7), layout='constrained')
         pca_for_plot_fadingplane = np.abs(rti_PCA_fadingplane[:, :])
         ax.imshow(pca_for_plot_fadingplane, origin='lower', aspect='auto', interpolation='none', extent=None)
         ax.set_xlabel('Time [correlations]')
         ax.set_ylabel('Gate [#]')
         ax.yaxis.set_major_locator(MaxNLocator(integer=True))
         ax.set_title('RTI, Averaging All Channels')
         plt.show()
```

```
(750000, 1)
```



```
In [79]: rti_for_det_fadingplane = reduce_xarray(rti_xarray_fadingplane,
                                                 dim_operations,
                                                 value_agg_func=np.abs) #Set up Data in the same format as in repository
         detections,img,list_dict_properties,peak_idx=PDA.detect_plane(rti=rti_for_det_fadingplane,
                                                                 start_samples=2000,
                                                                 detect_options={},
                                                                 optional_plots=['DBSCAN'])
```





In this example the plane detection algorithm accurately identified the plane object for the entire 15s of rti data provided. The short period is well within the value of eps, so the algorithm accurately categorizes each detection event as the same object.

Unfortunately, this plane is also relatively low to the ground, visible in the 2nd and 3rd gates. Typically, detection events in the first gate, due to ground interference, are classified as noise by the detection algorithm. In this case, they are within eps of detection events of the plane, and so are lumped into the same detected object.

Spikes in Background Noise

Small spikes in background noise is common in rti data, and can be due to a number of reasons. This is an example of the noise environment changing slightly with time, due to the plane. In this case, it does not result in errors in detection, but more drastic changes in the background environment do cause errors.

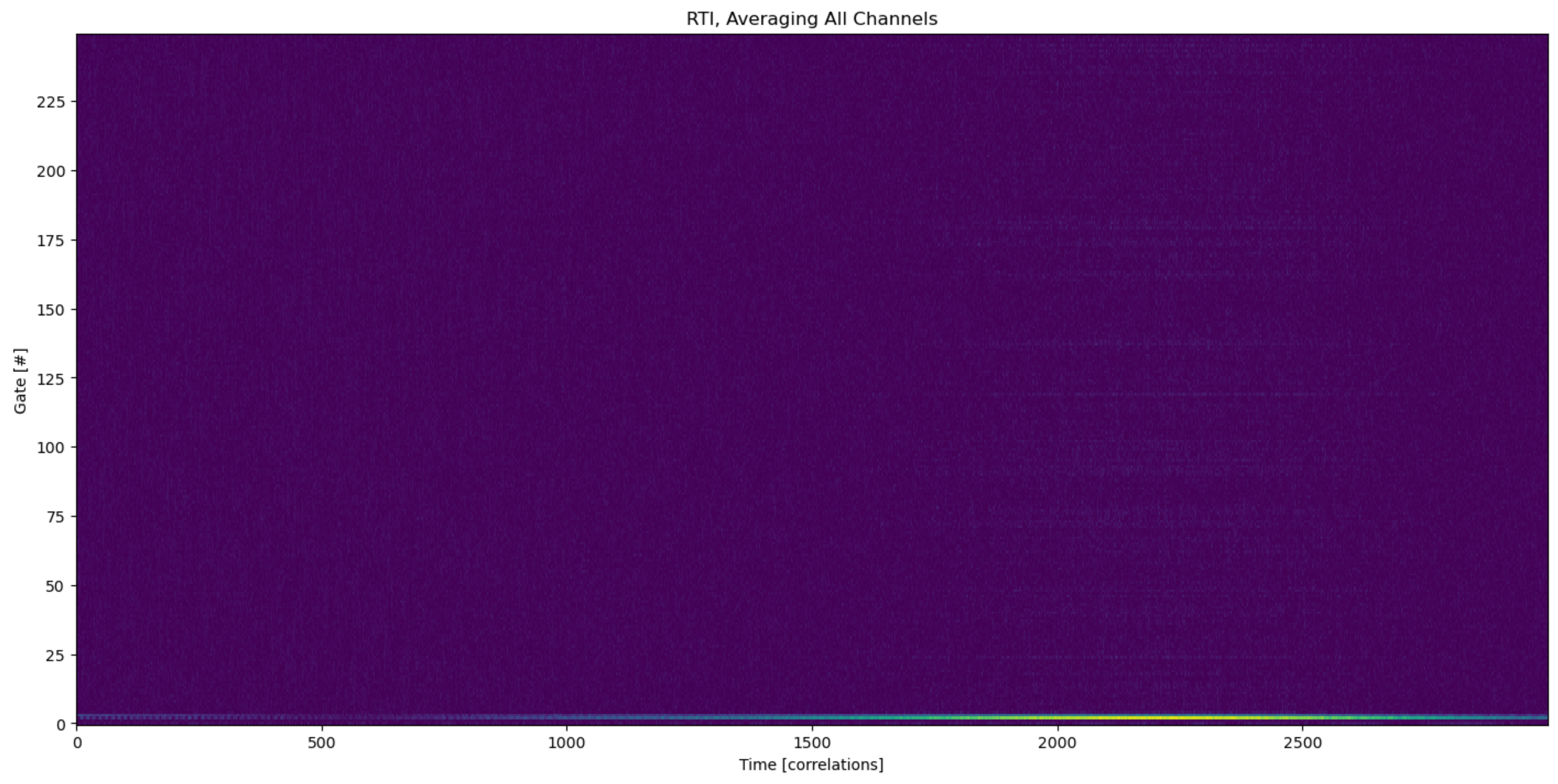
In [80]:

```python
file_name3 = 'RTI_mild_background_noise_15s.rti'
with open(f'{data_dir}/{file_name3}', "rb") as f:
        rti_xarray_backgroundinterfierence = pickle.load(f)
```

In [81]:

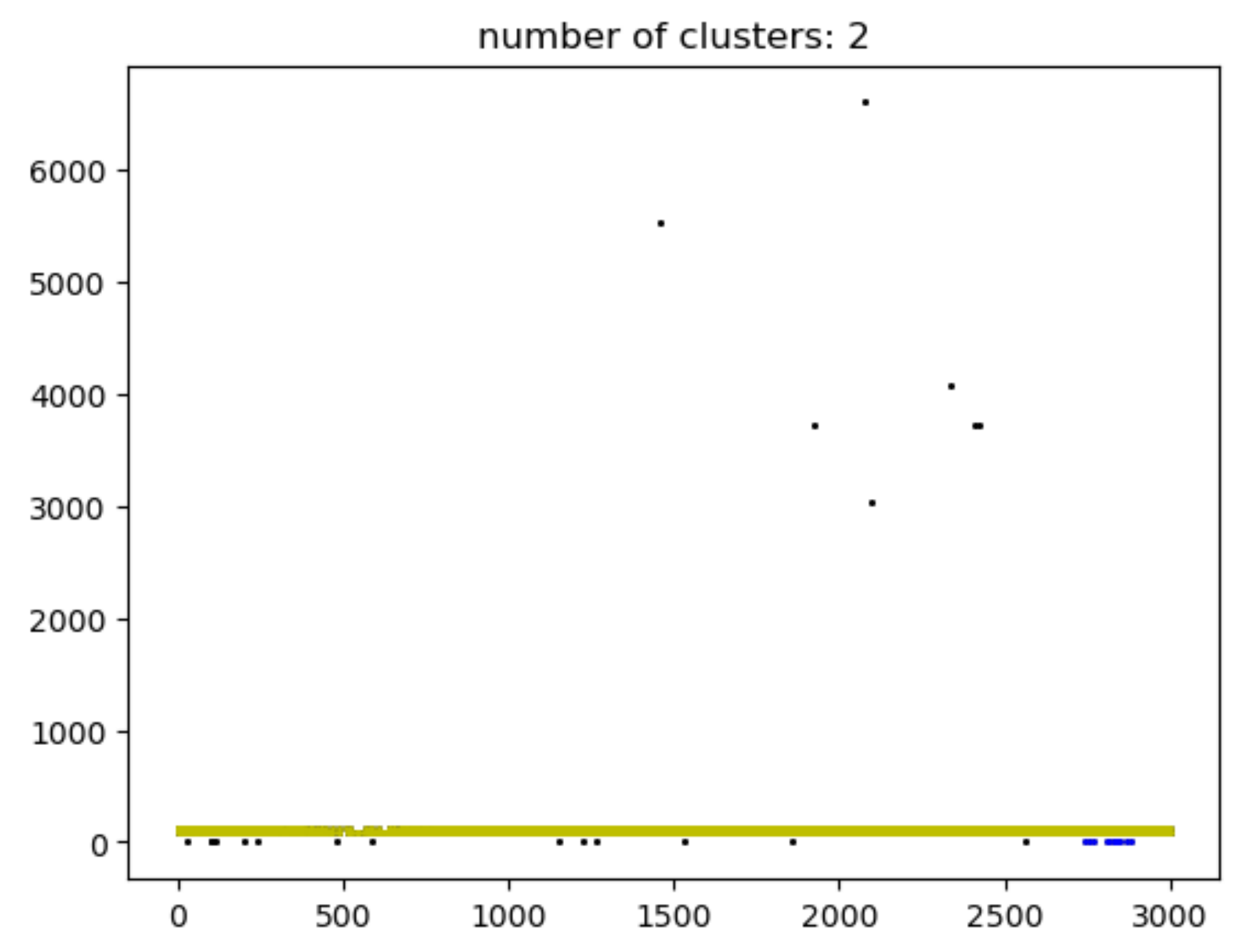
```python
rti_array_backgroundinterfierence = rti_xarray_backgroundinterfierence.values[:, 0, :250, :]
# Performing PCA
n_ch, n_gates, n_times = rti_array_backgroundinterfierence.shape
X = np.moveaxis(rti_array_backgroundinterfierence, 0, -1).reshape(-1, n_ch)
X = abs(X)
pca = PCA(n_components=1)
pca.fit(X)
rti_PCA_backgroundinterfierence = pca.fit_transform(X)
print(rti_PCA_backgroundinterfierence.shape)
rti_PCA_backgroundinterfierence = np.reshape(rti_PCA_backgroundinterfierence, (n_gates, n_times, 1)).squeeze()

# Plotting
fig, ax = plt.subplots(1, 1, figsize=(14, 7), layout='constrained')
pca_for_plot_backgroundinterfierence = np.abs(rti_PCA_backgroundinterfierence[:, :])
ax.imshow(pca_for_plot_backgroundinterfierence, origin='lower', aspect='auto', interpolation='none', extent=None)
ax.set_xlabel('Time [correlations]')
ax.set_ylabel('Gate [#]')
ax.yaxis.set_major_locator(MaxNLocator(integer=True))
ax.set_title('RTI, Averaging All Channels')
plt.show()
```

```
(750000, 1)
```



In [82]:

```python
rti_for_det_backgroundinterfierence = reduce_xarray(rti_xarray_backgroundinterfierence,
                                                    dim_operations,
                                                    value_agg_func=np.abs) #Set up Data in the same format as in repository
detections,img,list_dict_properties,peak_idx=PDA.detect_plane(rti=rti_for_det_backgroundinterfierence,
                                                            start_samples=2000,
                                                            detect_options={},
                                                            optional_plots=['DBSCAN'])
```



Despite the small fluctuations i the noise background, the detected plane object is very accurate. Although there are outliers in the data, possibly due to the background noise, they have accurately been classified as noise by the DBSCAN part of the algorithm.

The one prevailing issue is a false detected object due to ground interference in the first gate. In this case, several detection events in the first gate are by chance close enough together to be falsely identified as an individual cluster.

As discussed above, larger spikes in background noise can result in a faulty, or incomplete detection.

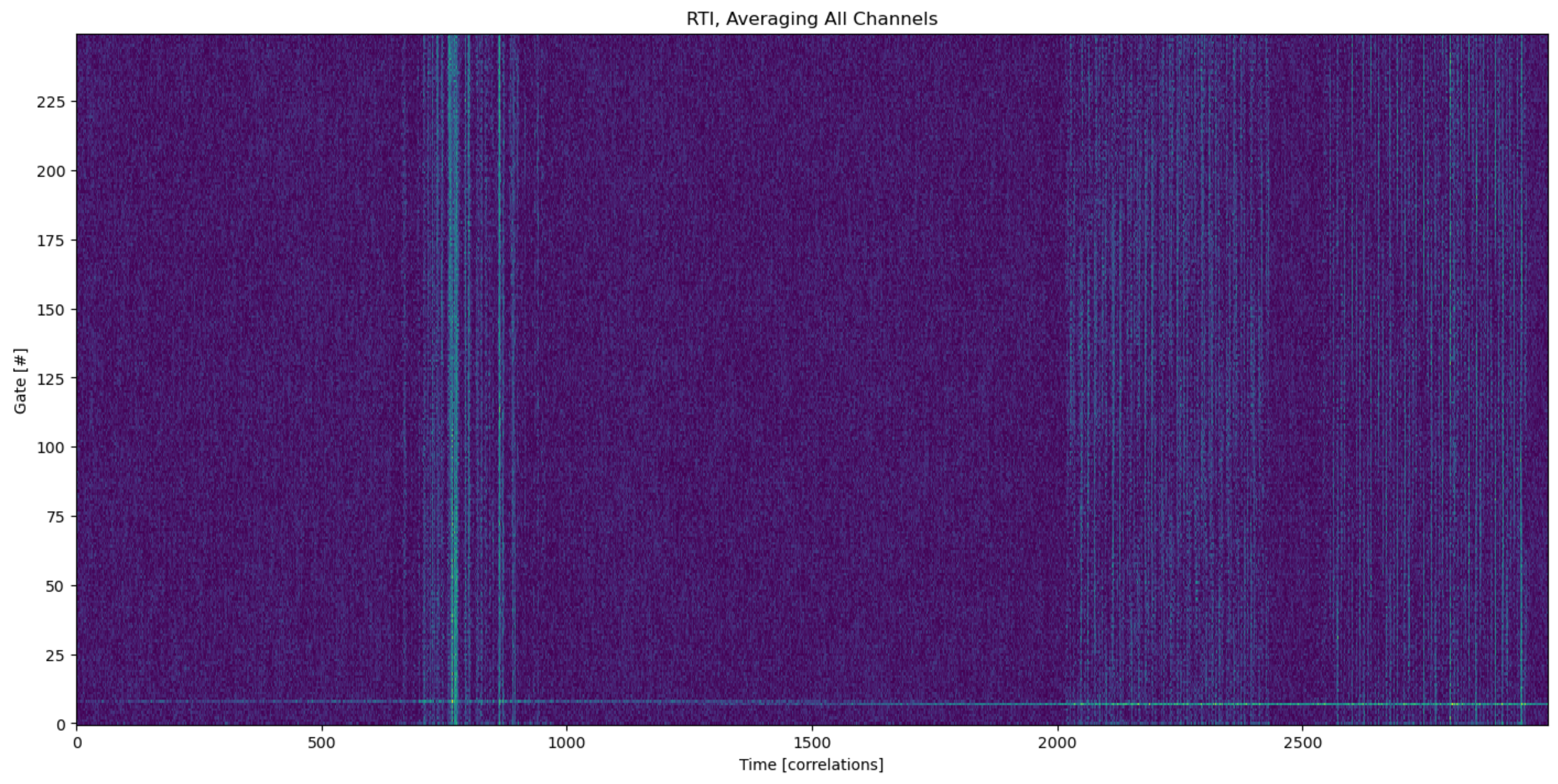
In [83]:

```python
file_name3 = 'RTI_zephyrtx3_zephyrrx2_1678326201_15s.rti'
with open(f'{data_dir}/{file_name3}', "rb") as f:
        rti_xarray_backgroundinterfierence2 = pickle.load(f)
```

In [84]:

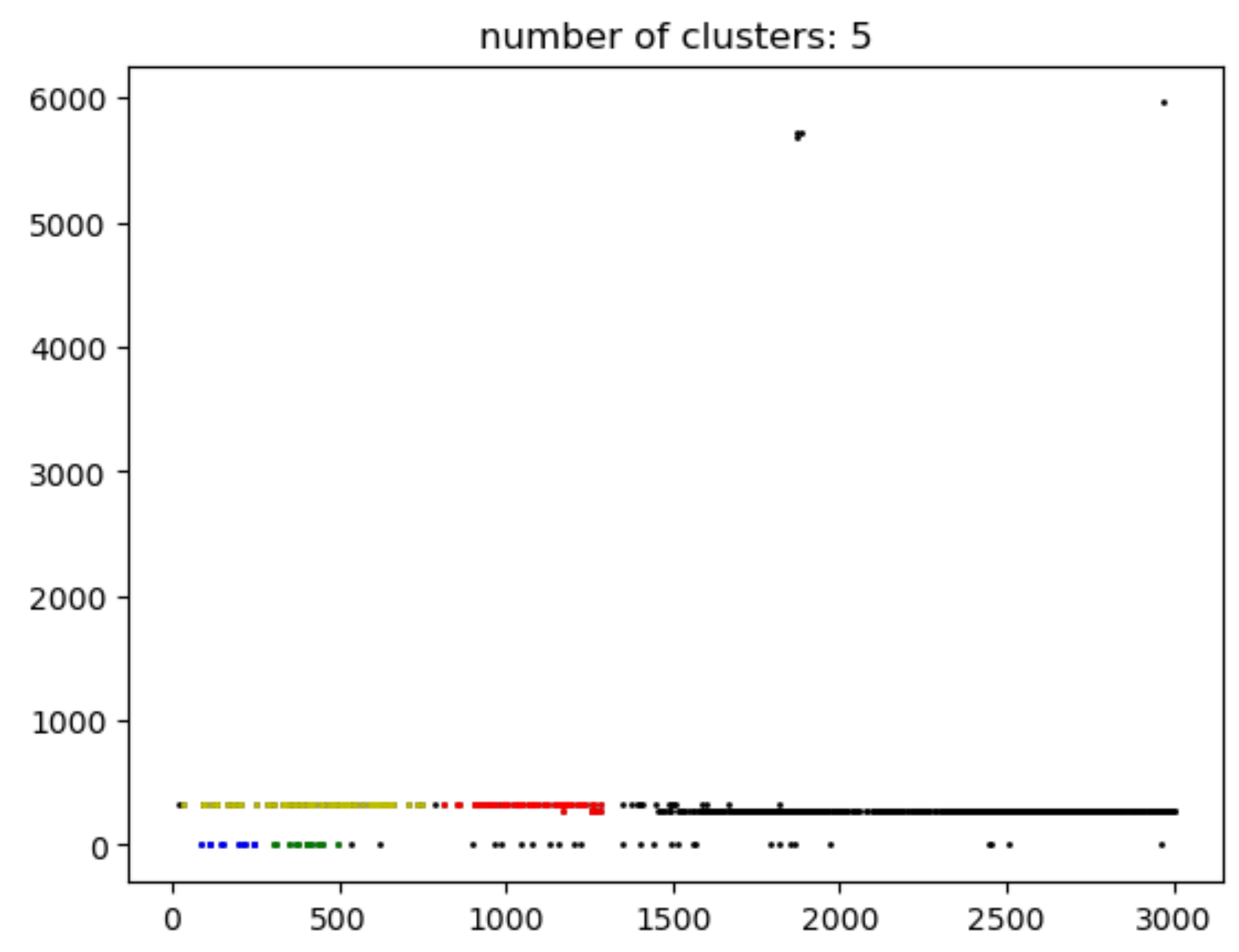
```python
rti_array_backgroundinterfierence2 = rti_xarray_backgroundinterfierence2.values[:, 0, :250, :]
# Performing PCA
n_ch, n_gates, n_times = rti_array_backgroundinterfierence2.shape
X = np.moveaxis(rti_array_backgroundinterfierence2, 0, -1).reshape(-1, n_ch)
X = abs(X)
pca = PCA(n_components=1)
pca.fit(X)
rti_PCA_backgroundinterfierence2 = pca.fit_transform(X)
print(rti_PCA_backgroundinterfierence2.shape)
rti_PCA_backgroundinterfierence2 = np.reshape(rti_PCA_backgroundinterfierence2, (n_gates, n_times, 1)).squeeze()

# Plotting
fig, ax = plt.subplots(1, 1, figsize=(14, 7), layout='constrained')
pca_for_plot_backgroundinterfierence2 = np.abs(rti_PCA_backgroundinterfierence2[:, :])
ax.imshow(pca_for_plot_backgroundinterfierence2, origin='lower', aspect='auto', interpolation='none', extent=None)
ax.set_xlabel('Time [correlations]')
ax.set_ylabel('Gate [#]')
ax.yaxis.set_major_locator(MaxNLocator(integer=True))
ax.set_title('RTI, Averaging All Channels')
plt.show()
```

```
(750000, 1)
```



In [85]:

```python
rti_for_det_backgroundinterfierence2 = reduce_xarray(rti_xarray_backgroundinterfierence2,
                                                    dim_operations,
                                                    value_agg_func=np.abs) #Set up Data in the same format as in repository
detections,img,list_dict_properties,peak_idx=PDA.detect_plane(rti=rti_for_det_backgroundinterfierence2,
                                                            start_samples=2000,
                                                            detect_options={},
                                                            optional_plots=['DBSCAN'])
```



The large noise spikes resulted in the detection algorithm being unable to properly correlate detections of a single plane to a single detected object. In the example above, the detection algorithm identifies 3 different clusters for a singular plane.

Additionally, there is still considerable ground interference, resulting in 2 incorrect detected objects in the first gate.

Other Potential issues

Despite the testing the plane detection algorithm has undergone, there are some scenarios that have not been tested, specifically, when there are more than one plane in the RTI data, or different sampling rate for the RTI.

It is possible that two or more planes will occupy the same or similar gate ranges throughout the RTI data. In this case it would be very difficult, or almost impossible to accurately identify them as individual objects. Planes which are further apart in the atmosphere, but still nearby in gate range may still be identified as the same detection object in the plane detection algorithm. Fortunately, these cases are very rare.

Additionally, if the sampling rate of the RTI data changed, the DBSCAN calibration could be entirely incorrect. This could cause both false detected objects, and inhibit the detection of existing objects among other problems.

Other Resources / other useful resources

Information on CFAR: https://www.youtube.com/watch?v=BEg29UuZk6c Information on DBSCAN: https://www.geeksforgeeks.org/dbscan-clustering-in-ml-density-based-clustering/ Python Documentation: https://docs.python.org/3/ Information on Zephyr as a whole and meteor radar: John Marino's thesis "On the Development of a Multistatic Meteor Radar Network for Investigating Dynamics of the Upper Atmosphere, Meteor Science, and Space Weather"

Further work

Despite it's success, the plane detection algorithm has error cases, and inconsistancies. Ideally, much of the further work relating to this function specifically would revolve around correcting those. Additionally, as further work is done on the auto calibration process, the plane detection algorithm should be changed and adjusted to meet the specific requirements of the calibration process.

Improvement on the algorithm itself can include: Further tuning the parameters associated with DBSCAN for more accurate overall detection, and prevent detection of meteors. Elimination of ground interference from data Additional post processing to catch errors such as singular planes being recorded as multiple detected objects, and correct existing errors Work to optimize the algorithm's run time, and computational requirements.

As further work is done on the auto calibration process, new requirements for the plane detection algorithm will be discovered. The Plane detection algorithm should be continued to be developed, and refined to meet these requirements throughout the rest of the process of automating the calibration process.